

Chapter 5.

Convolutional Neural Networks - CNNs

PhD. Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn



Convolutional Neural Network

Multimedia Data

Multimedia data refers to content that combines different forms of media, such as text, audio, images, videos, animations, and interactive elements. When multimedia data includes visual elements like images or videos, it directly interacts with our visual system.

The Visual Cortex

The visual cortex is the part of the brain responsible for processing visual information. It's located in the occipital lobe at the back of the brain

How Multimedia Data Interacts with the Visual Cortex

When we encounter visual multimedia data (images, videos, etc.), the process described above is initiated.

- **Images:** When viewing an image, the visual cortex analyzes its various components like shapes, colors, textures, and spatial relationships to help us understand what we are seeing. Different areas within the visual cortex work together to identify objects and interpret the scene.
- **Videos:** Videos present a stream of sequential images, adding the dimension of motion and time. The visual cortex not only processes each frame individually but also analyzes the changes between frames to perceive movement, track objects, and understand the sequence of events. The brain has to process a large amount of information in a short time when watching videos.

Different Types of Visual Multimedia

Different types of visual multimedia can engage the visual cortex in slightly different ways:

- **Static Images:** Primarily engage areas involved in shape and object recognition (ventral stream) and spatial awareness (dorsal stream).
- **Moving Images (Videos, Animations):** Strongly activate areas involved in motion detection (like V5) and the integration of visual information over time.
- **Interactive Visuals:** May further engage areas involved in spatial reasoning and motor planning if the user interacts with the visual elements.

Beyond the Visual Cortex

While the visual cortex is crucial for processing visual aspects of multimedia, it's important to remember that other parts of the brain are involved in a comprehensive multimedia experience. For example, audio components are processed in the auditory cortex, and text is processed in language areas of the brain. These different sensory inputs can be integrated to create a richer and more immersive experience.

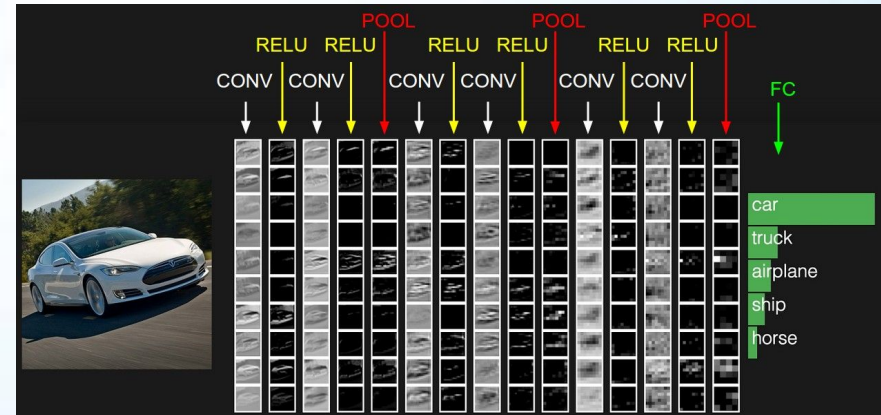
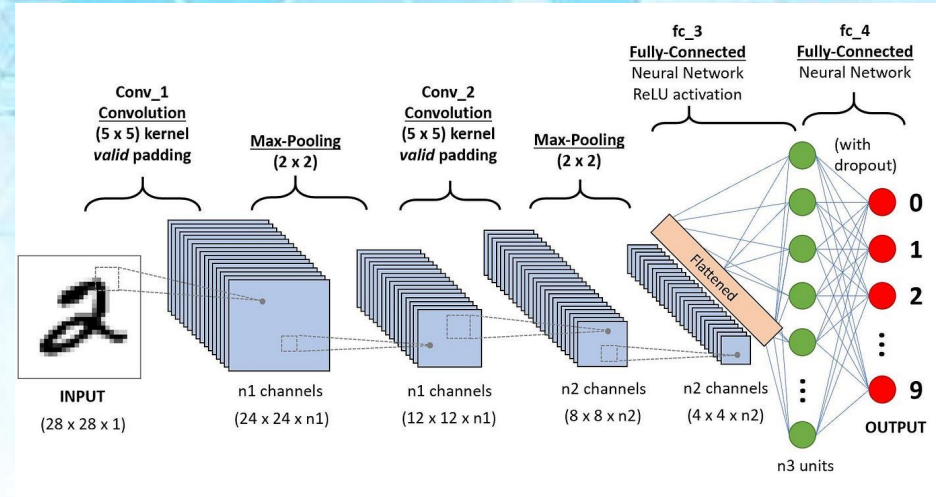
Convolutional Neural Network

Convolutional Neural Networks (CNNs) are a specialized type of artificial neural network particularly well-suited for processing structured grid data, such as images (which can be seen as a 2D grid of pixels). While they excel in image and video recognition, they are also used for tasks like natural language processing and audio analysis.

Traditional neural networks can struggle with image data due to:

- **High Dimensionality:** Images have a large number of pixels, leading to a massive number of input neurons and connections in a fully connected network. This makes training computationally expensive and prone to overfitting.
- **Spatial Relationships:** Traditional networks treat each pixel independently, ignoring the crucial spatial relationships between neighboring pixels that form meaningful patterns (edges, corners, textures, objects).
- **Translation Invariance:** An object in an image can appear in different locations. A fully connected network would need to learn to recognize the object in every possible position separately.

CNNs address these limitations through their unique architecture and key components.



Convolutional Neural Network

Applications of CNNs:

CNNs have revolutionized many fields and are used in a wide range of applications, including:

- **Image Classification:** Identifying the objects present in an image (e.g., cats vs. dogs, different types of cars).
- **Object Detection:** Locating and identifying multiple objects within an image (e.g., detecting cars, pedestrians, and traffic signs in a scene).
- **Image Segmentation:** Dividing an image into different regions based on the objects or categories present.
- **Facial Recognition:** Identifying individuals based on their facial images.
- **Medical Image Analysis:** Detecting diseases, segmenting organs, and analyzing medical scans.
- **Natural Language Processing:** For tasks like text classification, sentiment analysis, and machine translation (though Recurrent Neural Networks (RNNs) and Transformers are more commonly used for sequential data).
- **Video Analysis:** Understanding and interpreting video content, such as action recognition and video surveillance.

Advantages of CNNs:

- **Automatic Feature Extraction:** CNNs can automatically learn relevant features from raw image data, eliminating the need for manual feature engineering.
- **Spatial Hierarchy of Features:** The layered architecture allows CNNs to learn hierarchical representations of features, from simple edges and textures in early layers to complex object parts and entire objects in later layers.
- **Translation Invariance:** Pooling layers and the convolutional operation with shared weights make CNNs more robust to the location of objects in an image.
- **Parameter Sharing:** The same filter is applied across the entire input image, which significantly reduces the number of parameters compared to fully connected networks, making training more efficient and less prone to overfitting.

Disadvantages of CNNs:

- **Data Dependency:** CNNs typically require large amounts of labeled data to train effectively.
- **Computational Cost:** Training deep CNNs can be computationally expensive and time-consuming, requiring significant hardware resources (like GPUs).
- **Black Box Nature:** It can be difficult to interpret exactly why a CNN makes a particular prediction.
- **Sensitivity to Hyperparameters:** The performance of a CNN can be highly sensitive to the choice of hyperparameters (e.g., number of layers, filter sizes, learning rate).

Convolutional Neural Network

Typical CNN Architecture:

A typical CNN architecture consists of a sequence of these layers:

Input Image -> Convolutional Layer -> Pooling Layer -> Convolutional Layer -> Pooling Layer -> ... -> Fully Connected Layer -> Output

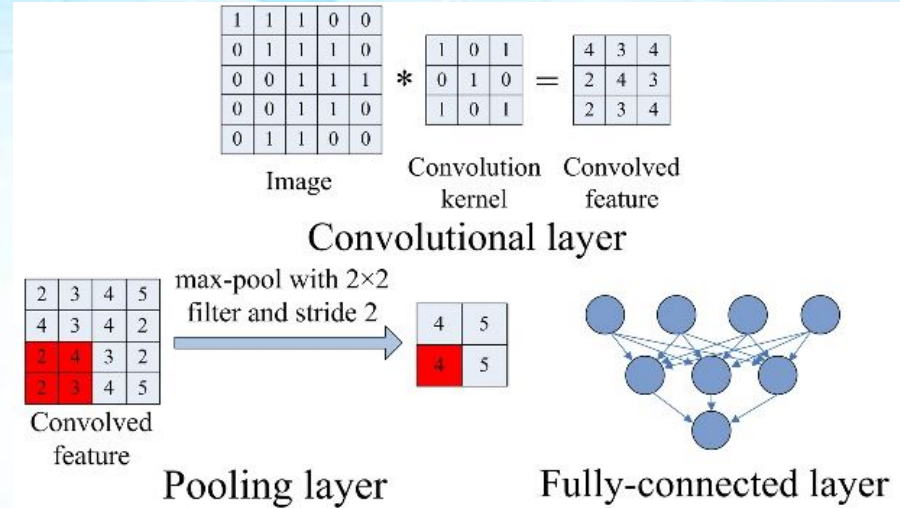
The exact number and arrangement of these layers depend on the specific problem and dataset.

How CNNs Learn:

CNNs learn through a process called **backpropagation**, similar to other neural networks.

1. **Forward Pass:** An input image is fed through the network, and the output (e.g., predicted class probabilities) is calculated.
2. **Loss Function:** The difference between the predicted output and the actual label (ground truth) is measured using a loss function.
3. **Backpropagation:** The gradients of the loss function with respect to the network's weights are calculated and propagated backward through the network.
4. **Weight Update:** The weights of the filters and fully connected layers are adjusted based on these gradients using an optimization algorithm (e.g., Adam, SGD) to minimize the loss.

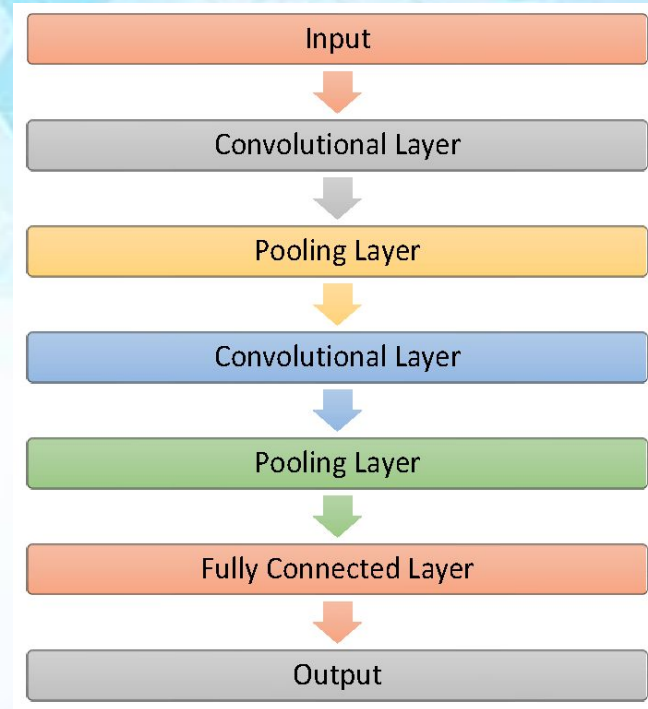
This process is repeated over many iterations (epochs) on a large dataset, allowing the CNN to learn the optimal weights for the given task.



Convolutional Neural Network

The key components of CNNs:

1. **Convolutional Layers:** These layers apply convolution operations on the input data using filters (kernels) that slide over the input. They capture spatial hierarchies and patterns in the data.
2. **Activation Functions:** After convolution, an activation function (such as ReLU, sigmoid, or tanh) is applied to introduce non-linearity, allowing the network to learn complex patterns.
3. **Pooling Layers:** These layers down-sample the feature maps, reducing their spatial dimension and the number of parameters, which helps to prevent overfitting. Common pooling methods include max pooling and average pooling.
4. **Fully Connected Layers:** At the end of the CNN, fully connected layers interpret the features extracted by the convolutional and pooling layers to make final predictions. Each neuron in a fully connected layer is connected to every neuron in the previous layer.
5. **Normalization Layers:** Techniques like Batch Normalization may be used to stabilize and accelerate training by normalizing the inputs to layers within the network.
6. **Dropout Layers:** These layers randomly set a fraction of the neurons to zero during training to prevent overfitting and improve generalization.
7. **Input Layer:** This layer receives the image data, which can be in various dimensions (like height, width, and color channels).
8. **Output Layer:** This layer provides the final output of the CNN, often using a softmax activation function for classification tasks to output probabilities for each class.



Implementation CNNs with PyTorch

PyTorch provides the `torch.nn` module, which contains all the necessary building blocks for creating neural networks, including CNNs. Here are some key components:

- **`torch.nn.Conv2d`:** This class implements the 2D convolution operation. Its main parameters are:
 - `in_channels`: The number of input channels (e.g., 1 for grayscale images, 3 for RGB images).
 - `out_channels`: The number of output channels (number of filters).
 - `kernel_size`: The size of the convolutional kernel (e.g., 3 for a 3x3 kernel). Can be an integer for a square kernel or a tuple (`height`, `width`) for a rectangular kernel.
 - `stride`: The stride of the convolution operation (how many pixels the kernel moves in each step). Defaults to 1.
 - `padding`: Padding added to the input. Can be an integer for symmetric padding or a tuple (`padding_height`, `padding_width`).
- **`torch.nn.MaxPool2d`:** This class implements the 2D max pooling operation. Its main parameters are:
 - `kernel_size`: The size of the pooling window.
 - `stride`: The stride of the pooling operation. If `None`, it defaults to `kernel_size`.
- **`torch.nn.Linear`:** This class implements a linear (fully connected) layer. Its main parameters are:
 - `in_features`: The number of input features.
 - `out_features`: The number of output features.
- **Activation Functions:** PyTorch provides various activation functions in `torch.nn`, such as:
 - `torch.nn.ReLU`: Rectified Linear Unit.
 - `torch.nn.Sigmoid`: Sigmoid function.
 - `torch.nn.Tanh`: Hyperbolic tangent function.
- **`torch.nn.Module`:** This is the base class for all neural network modules in PyTorch. You will define your custom CNN architecture as a subclass of `torch.nn.Module`.

Implementation CNNs with PyTorch

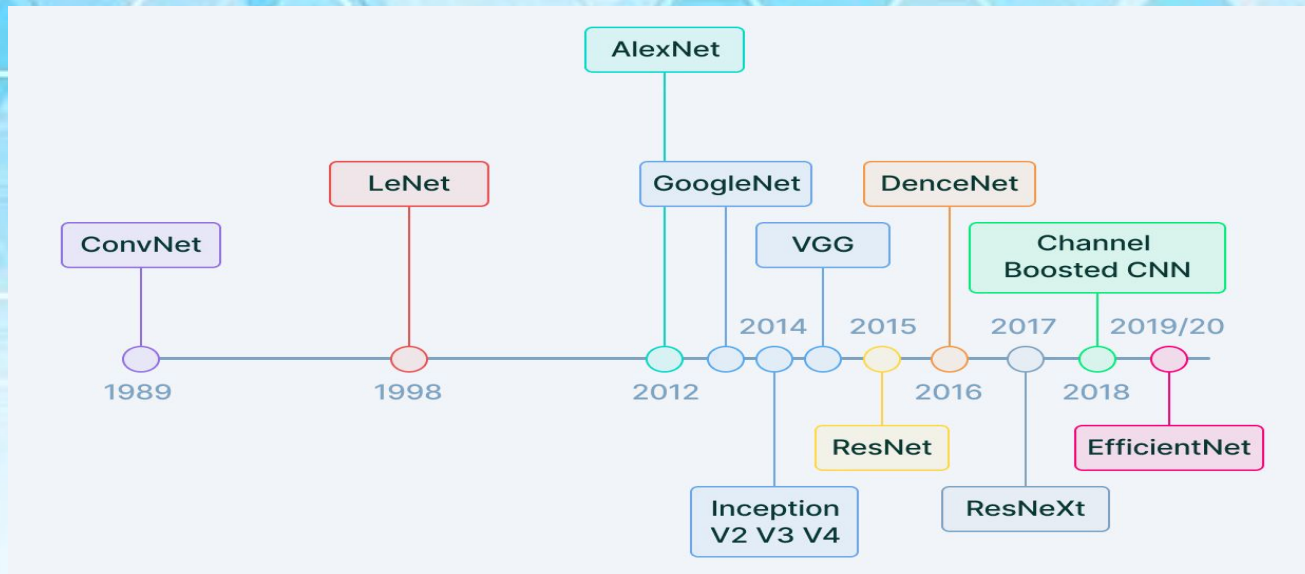
```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torch.optim as optim
5 import torchvision
6 import torchvision.transforms as transforms
7 from torch.utils.data import DataLoader
8 # Define the CNN model
9 class SimpleCNN(nn.Module):
10     def __init__(self, num_classes=10):
11         super(SimpleCNN, self).__init__()
12         self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
13         self.relu1 = nn.ReLU()
14         self.maxpool1 = nn.MaxPool2d(kernel_size=2, stride=2)
15         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
16         self.relu2 = nn.ReLU()
17         self.maxpool2 = nn.MaxPool2d(kernel_size=2, stride=2)
18         self.fc1 = nn.Linear(64 * 7 * 7, 128)
19         self.relu3 = nn.ReLU()
20         self.fc2 = nn.Linear(128, num_classes)
21
22     def forward(self, x):
23         x = self.conv1(x)
24         x = self.relu1(x)
25         x = self.maxpool1(x)
26         x = self.conv2(x)
27         x = self.relu2(x)
28         x = self.maxpool2(x)
29         x = x.view(-1, 64 * 7 * 7)
30         x = self.fc1(x)
31         x = self.relu3(x)
32         x = self.fc2(x)
33         return x
```

References:

https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Common Convolutional Neural Network (CNN) architectures



Model	Size	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth
VGG16	528 MB	0.713	0.901	138,357,544	23
InceptionV3	92 MB	0.779	0.937	23,851,784	159
ResNet50	98 MB	0.749	0.921	25,636,712	-
Xception	88 MB	0.790	0.945	22,910,480	126
InceptionResNetV2	215 MB	0.803	0.953	55,873,736	572
ResNeXt50	96 MB	0.777	0.938	25,097,128	-

The top-1 and top-5 accuracy refers to the model's performance on the ImageNet validation dataset.

Depth refers to the topological depth of the network. This includes activation layers, batch normalization layers etc.

Common Convolutional Neural Network (CNN) architectures

LeNet-5 (1998):

- **Key Innovation:** One of the earliest successful CNN architectures, primarily used for handwritten digit recognition.
- **Architecture:** Consists of convolutional layers, pooling layers (originally average pooling), and fully connected layers. It has a relatively simple structure with a small number of parameters.
- **Significance:** Laid the foundation for modern CNNs and demonstrated their effectiveness for image recognition tasks.

AlexNet (2012):

- **Key Innovation:** Popularized CNNs in the deep learning era by winning the ImageNet Large Scale Visual Recognition Challenge (ILSVRC).
- **Architecture:** Deeper and wider than LeNet-5, it introduced concepts like ReLU activation functions, dropout for regularization, and the use of multiple GPUs for training. It also used max pooling.
- **Significance:** Showed the potential of deep CNNs for complex image classification tasks and sparked a surge of research in the field.

VGGNet (2014):

- **Key Innovation:** Focused on the effect of network depth, using a very uniform architecture with small (3x3) convolutional filters throughout the entire network.
- **Architecture:** Comes in different variations (VGG16, VGG19) with a large number of convolutional layers followed by fully connected layers.
- **Significance:** Demonstrated that increasing the depth of the network can improve performance significantly. Its simple and consistent structure made it widely adopted.

<https://pytorch.org/vision/main/models/vgg.html>

GoogLeNet (Inception v1) (2014):

- **Key Innovation:** Introduced the "Inception module," which allows the network to choose between different convolutional filter sizes (1x1, 3x3, 5x5) and pooling operations in parallel, making the network wider and more efficient.
- **Architecture:** Consists of multiple Inception modules stacked on top of each other, with occasional max-pooling layers. It also included auxiliary classifiers to help with training very deep networks.
- **Significance:** Showed that network architecture could be more complex and efficient than simply stacking convolutional layers.

Common CNN architectures

ResNet (Residual Networks) (2015):

- **Key Innovation:** Addressed the vanishing gradient problem in very deep networks by introducing "residual connections" or "skip connections." These connections allow the gradient to flow more directly through the network, enabling the training of much deeper networks.
- **Architecture:** Composed of residual blocks, where the output of a block is added to its input. This allows the network to learn "residual mappings" instead of directly trying to learn the underlying mapping. Different versions exist (*ResNet-18*, *ResNet-50*, *ResNet-101*, etc.) with varying depths.
- **Significance:** Enabled the training of extremely deep networks (hundreds or even thousands of layers), leading to significant improvements in performance on various tasks.

https://pytorch.org/hub/pytorch_vision_resnet/

<https://viso.ai/deep-learning/resnet-residual-neural-network/>

DenseNet (Densely Connected Convolutional Networks) (2017):

- **Key Innovation:** Further emphasized feature reuse by connecting each layer to every other layer in a "dense block."
- **Architecture:** Consists of dense blocks where the feature maps of all preceding layers are concatenated and used as inputs to the subsequent layers. Transition layers are used between dense blocks to reduce the dimensionality.
- **Significance:** Improved gradient flow and feature propagation throughout the network, leading to state-of-the-art performance with fewer parameters compared to ResNet.

<https://github.com/pytorch/vision/blob/main/torchvision/models/densenet.py>

<https://www.louisbouchard.ai/densenet-explained/>

<https://github.com/liuzhuang13/DenseNet?ref=louisbouchard.ai>

Common CNN architectures

MobileNet

(2017

onwards):

- **Key Innovation:** Designed specifically for mobile and embedded vision applications with a focus on efficiency (low latency and small model size).
- **Architecture:** Utilizes depthwise separable convolutions, which significantly reduce the number of parameters and computational cost compared to ¹ standard convolutions. Different versions (MobileNetV1, MobileNetV2, MobileNetV3) introduce further optimizations and architectural improvements.
- **Significance:** Enabled the deployment of deep learning models on resource-constrained devices.
<https://github.com/Shubhamai/pytorch-mobilenet>
<https://github.com/jmjeon2/MobileNet-Pytorch>

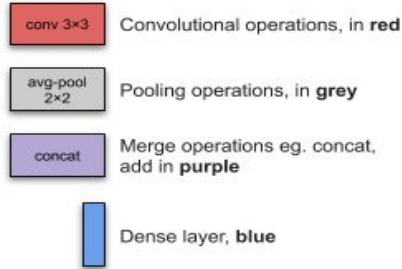
EfficientNet

(2019):

- **Key Innovation:** Introduced a systematic way to scale the dimensions of a CNN (depth, width, and resolution) in a balanced manner to achieve better performance and efficiency.
- **Architecture:** Starts with a baseline network (EfficientNet-B0) and then scales it up using a compound scaling method that jointly optimizes all three dimensions based on a scaling coefficient. Different models (EfficientNet-B1 to B7) are created by using different scaling coefficients.
- **Significance:** Achieved state-of-the-art accuracy on ImageNet with significantly fewer parameters and FLOPS compared to previous models.
<https://github.com/lukemelas/EfficientNet-PyTorch>
https://pytorch.org/hub/nvidia_deeplearningexamples_efficientnet/
<https://viso.ai/deep-learning/efficientnet/>

Common Convolutional Neural Network (CNN) architectures

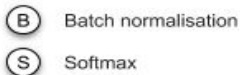
Layers



Activation Functions

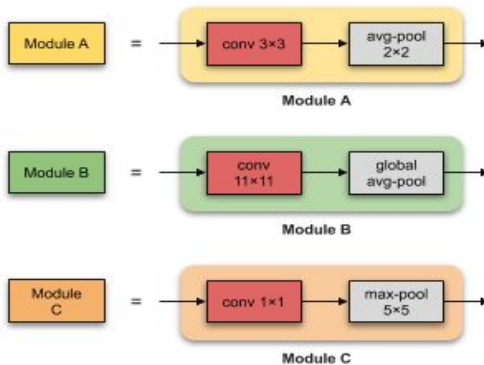


Other Functions

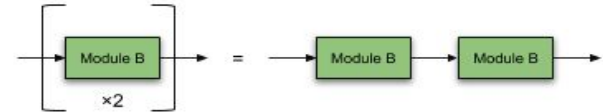


Modules/Blocks

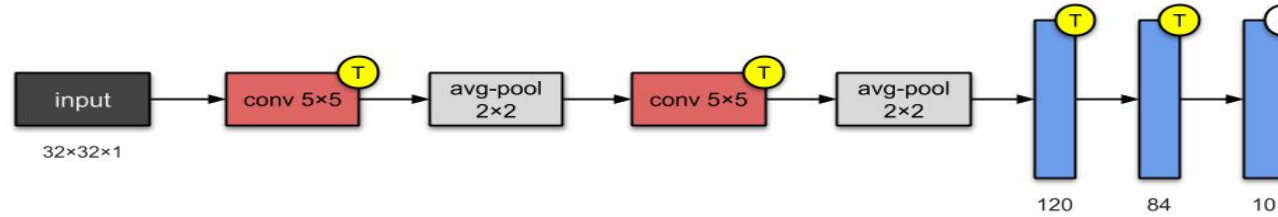
Modules (groups of convolutional, pooling and merge operations), in **yellow**, **green**, or **orange**. The operations that make up these modules will also be shown.



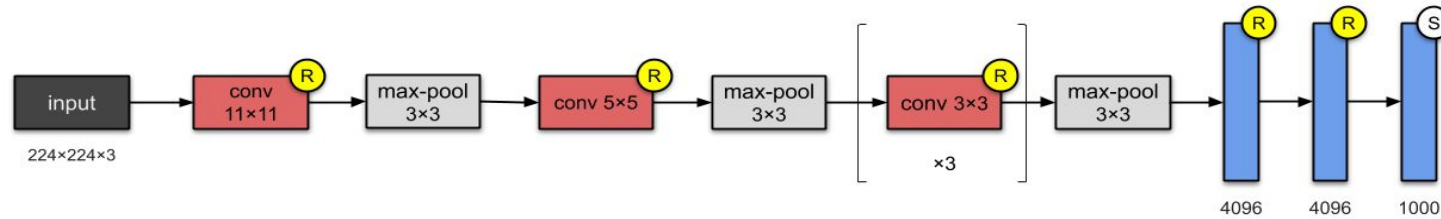
Repeated layers or modules/blocks



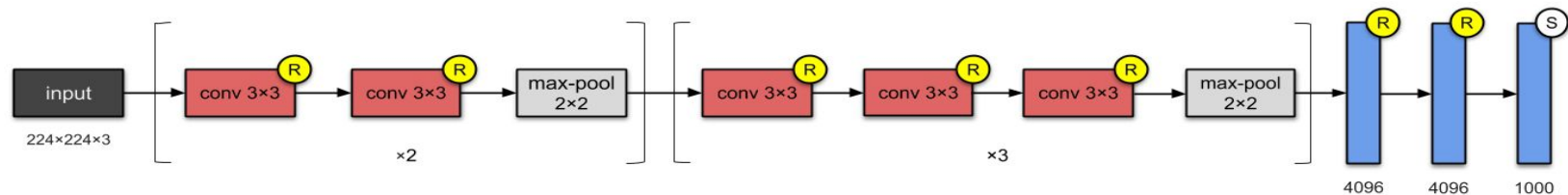
1. LeNet-5 (1998)



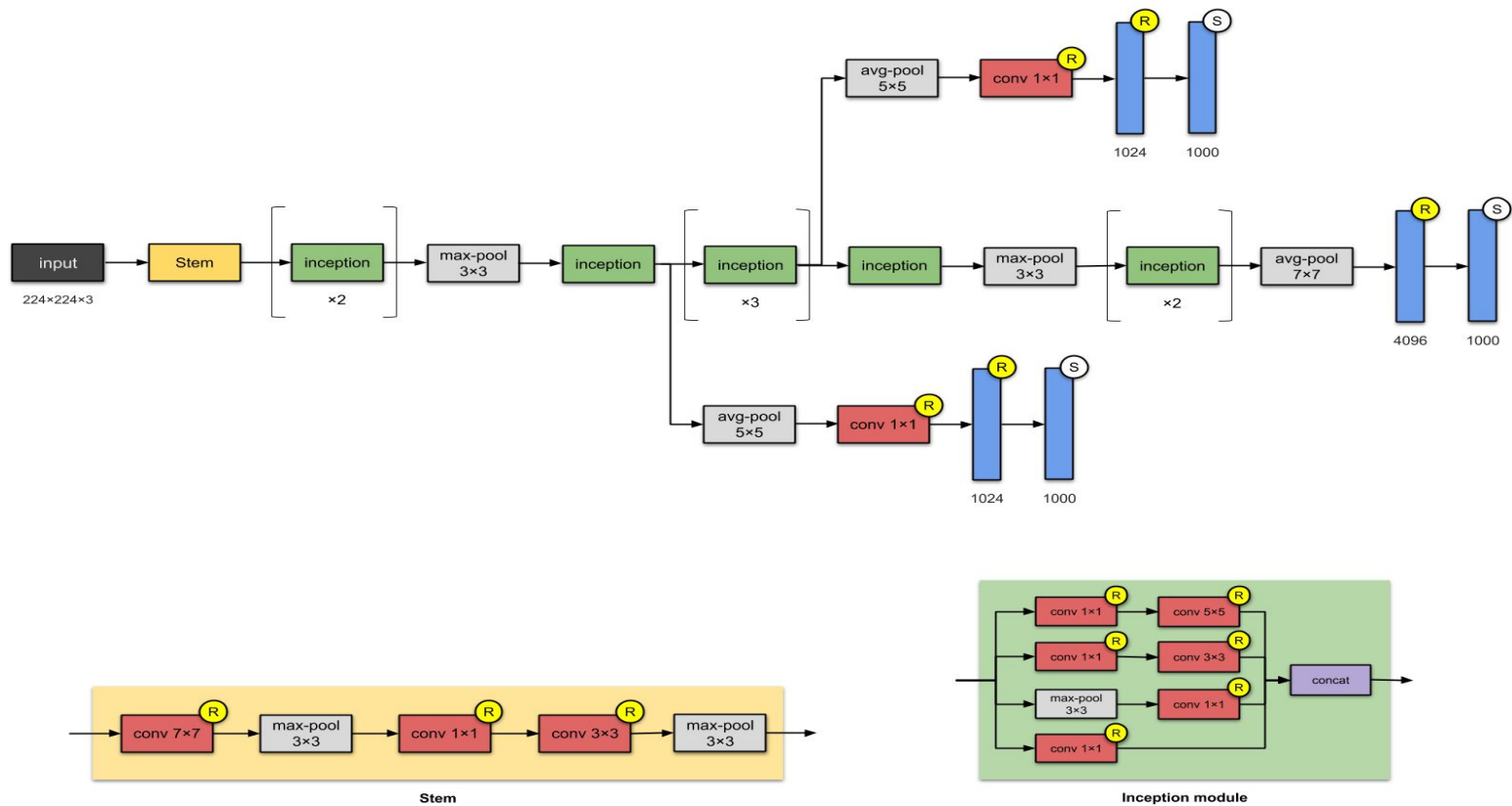
2. AlexNet (2012)



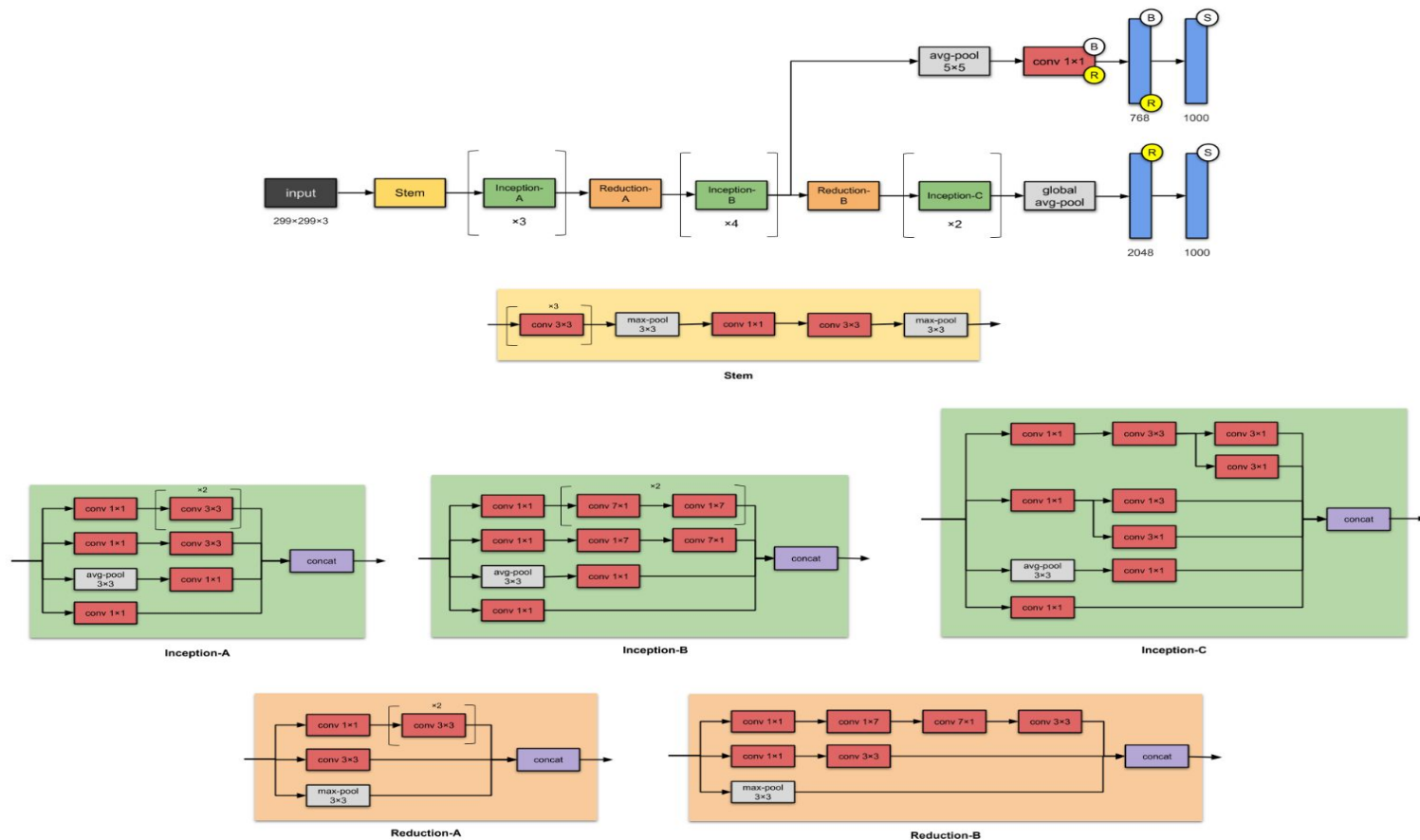
3. VGG-16 (2014)



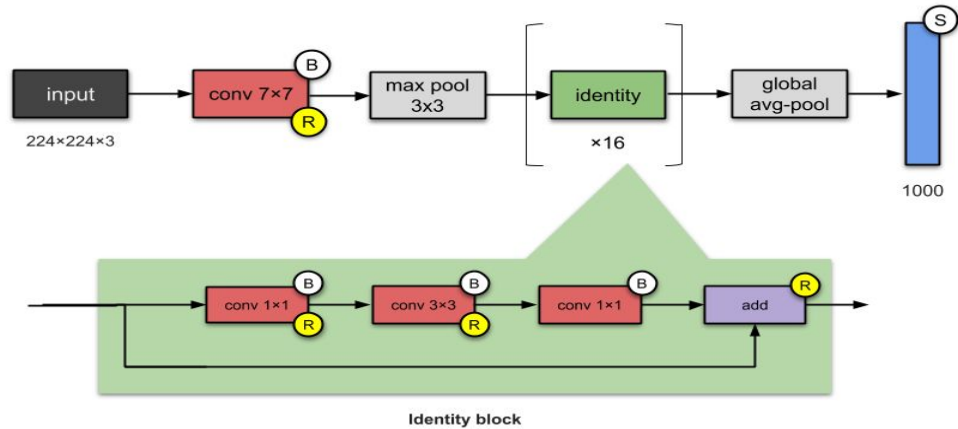
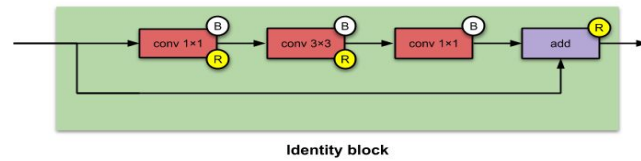
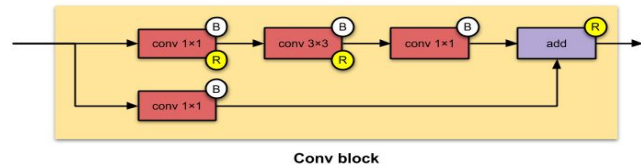
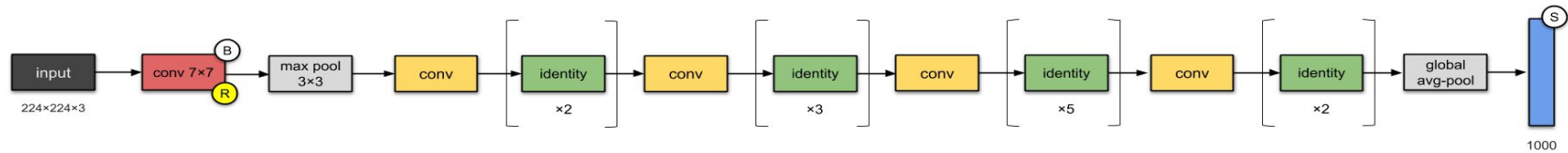
4. Inception-v1 (2014)



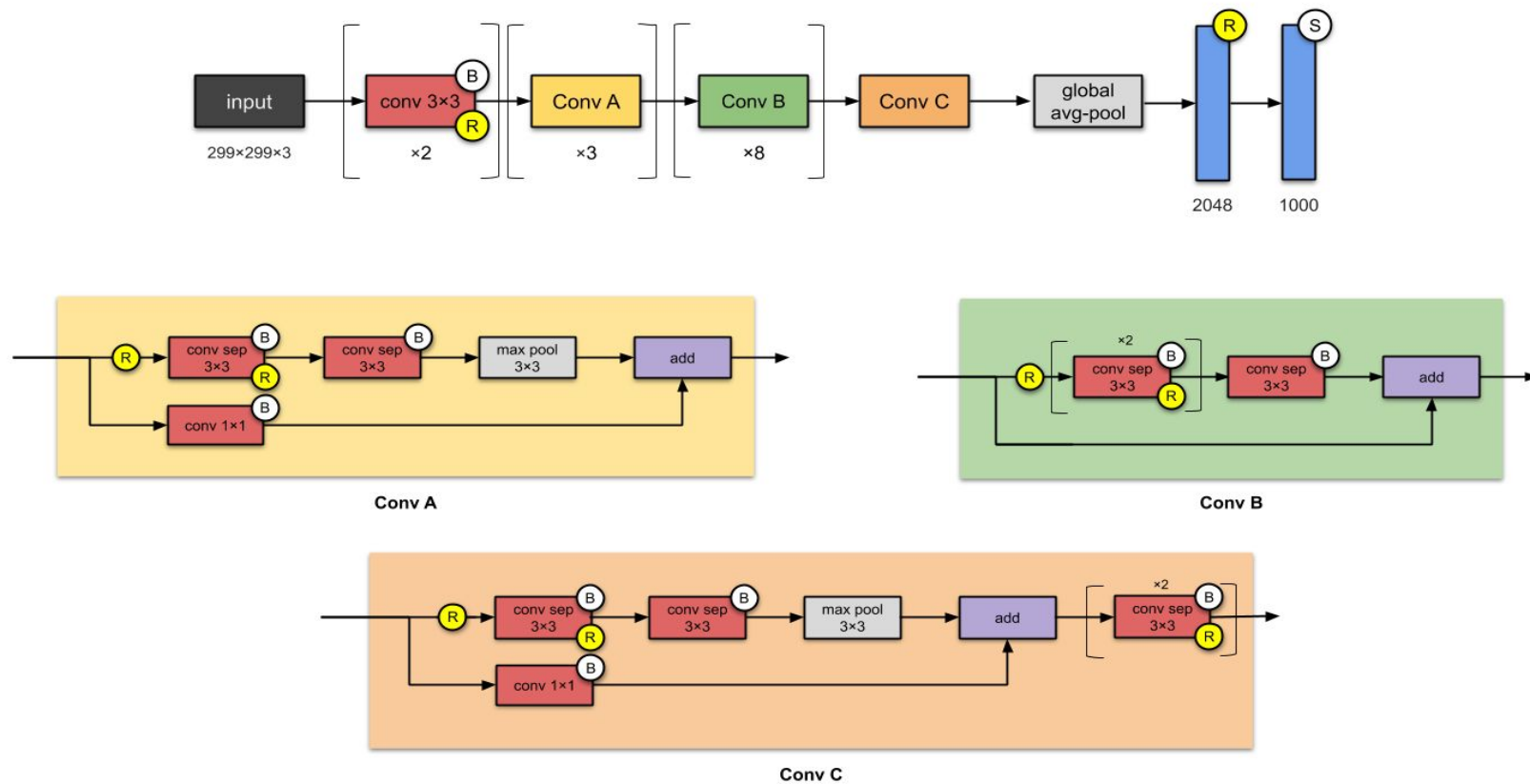
5. Inception-v3 (2015)



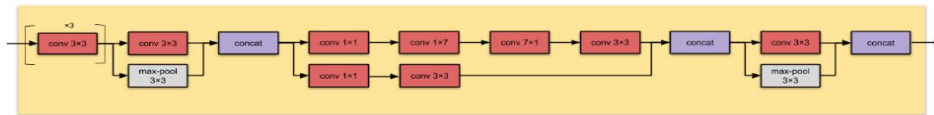
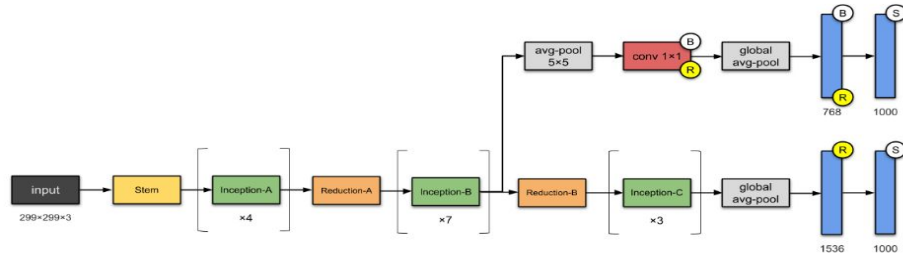
6. ResNet-50 (2015)



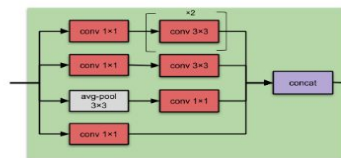
7. Xception (2016)



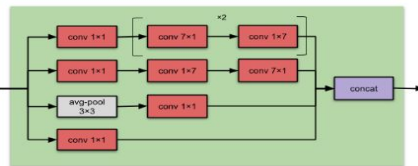
8. Inception-v4 (2016)



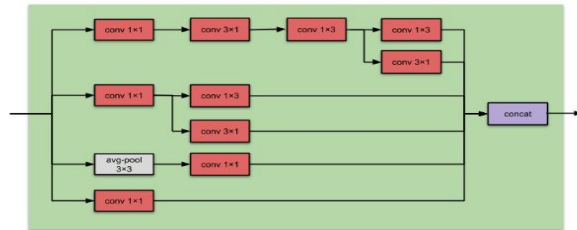
Stem



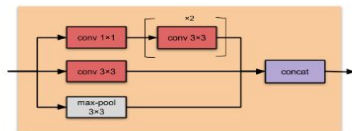
Inception-A



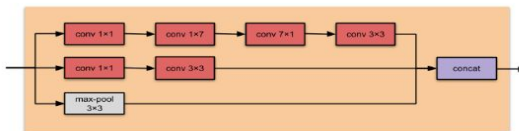
Inception-B



Inception-C

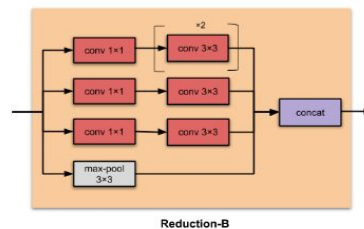
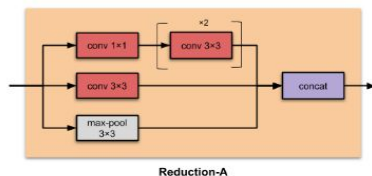
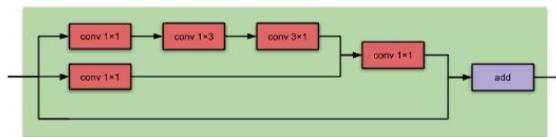
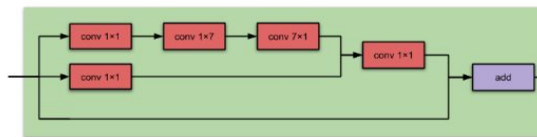
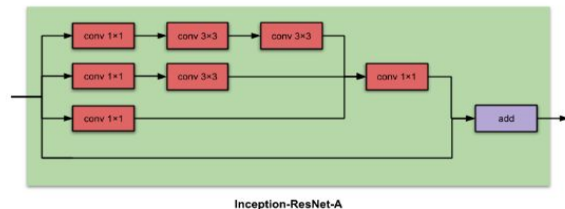
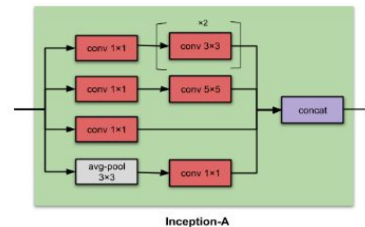
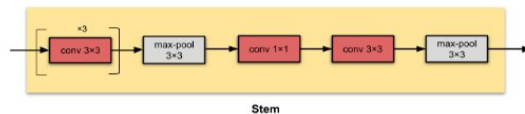
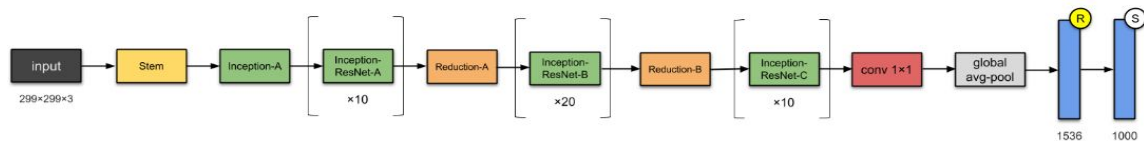


Reduction-A

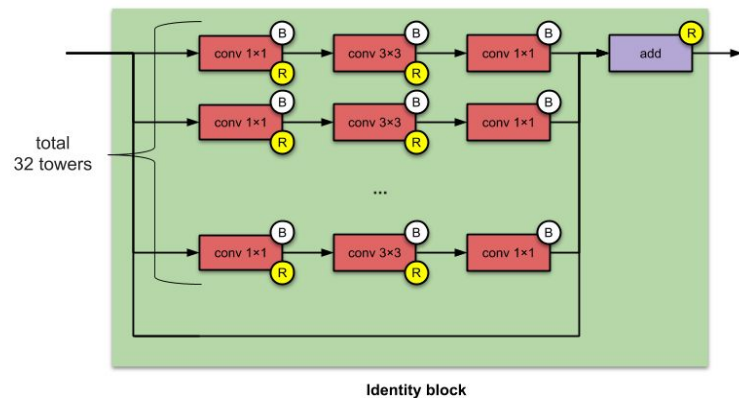
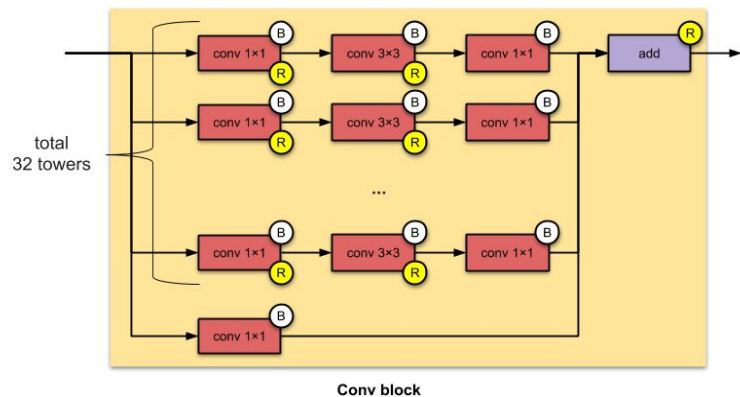
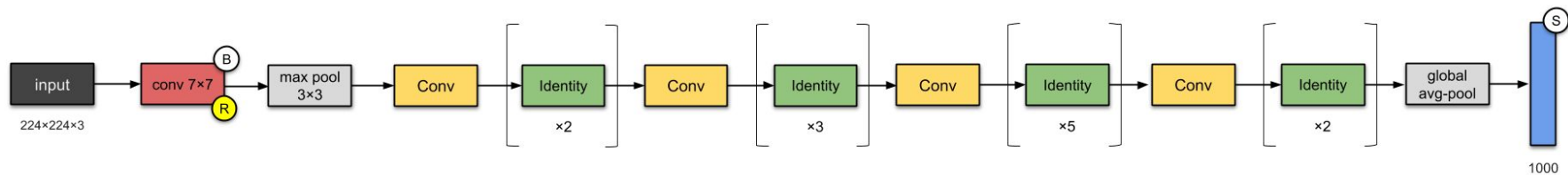


Reduction-B

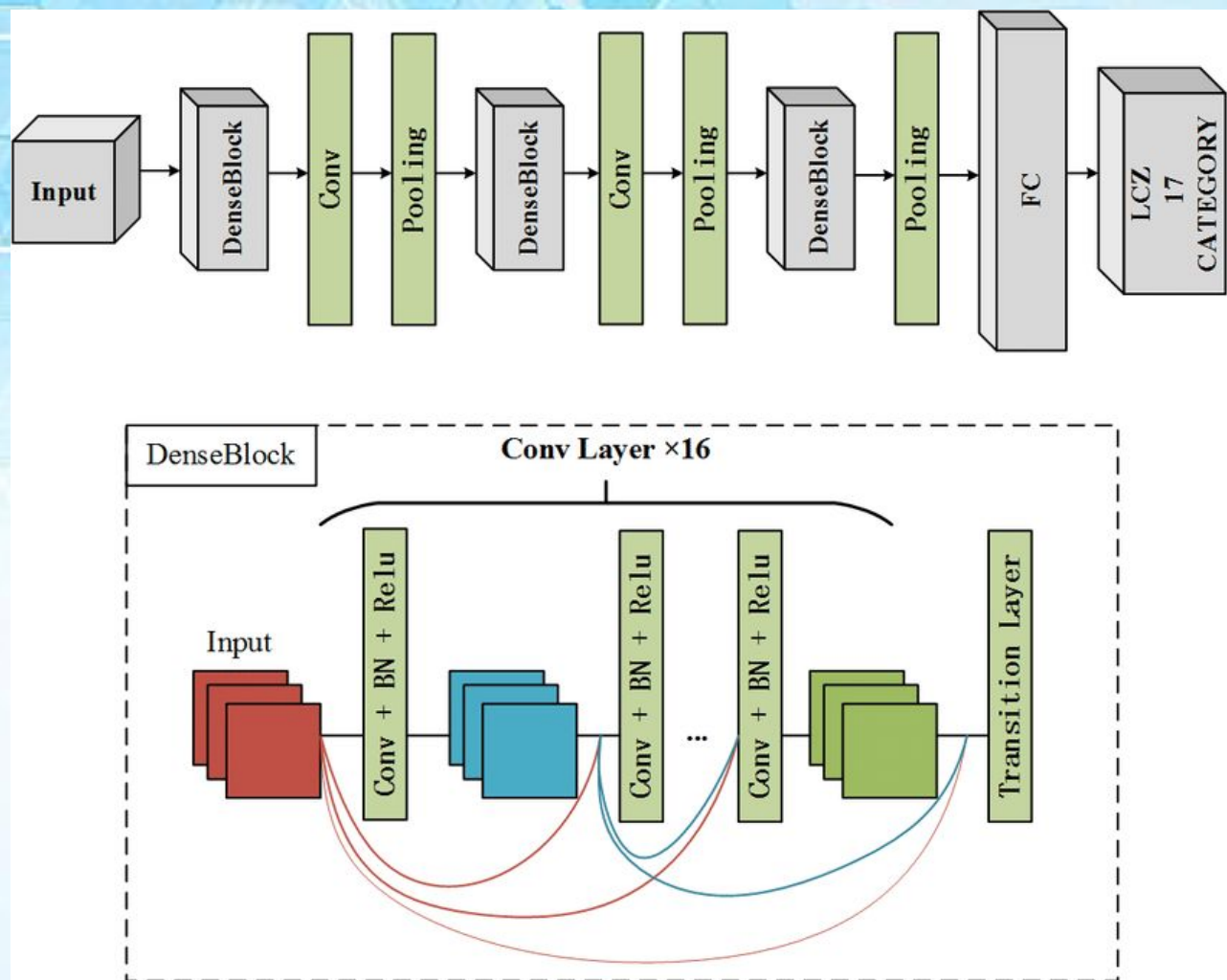
9. Inception-ResNet-V2 (2016)



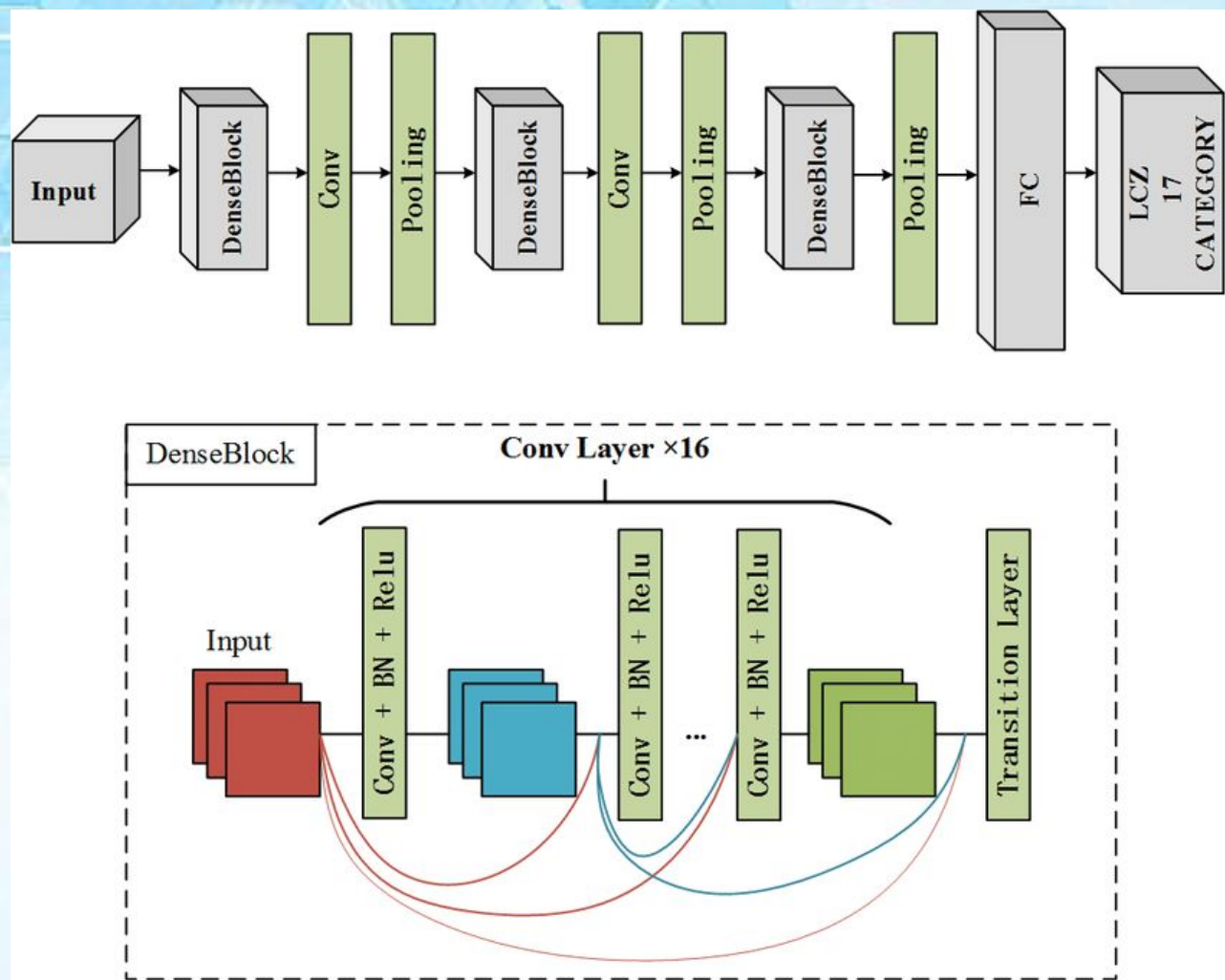
10. ResNeXt-50 (2017)



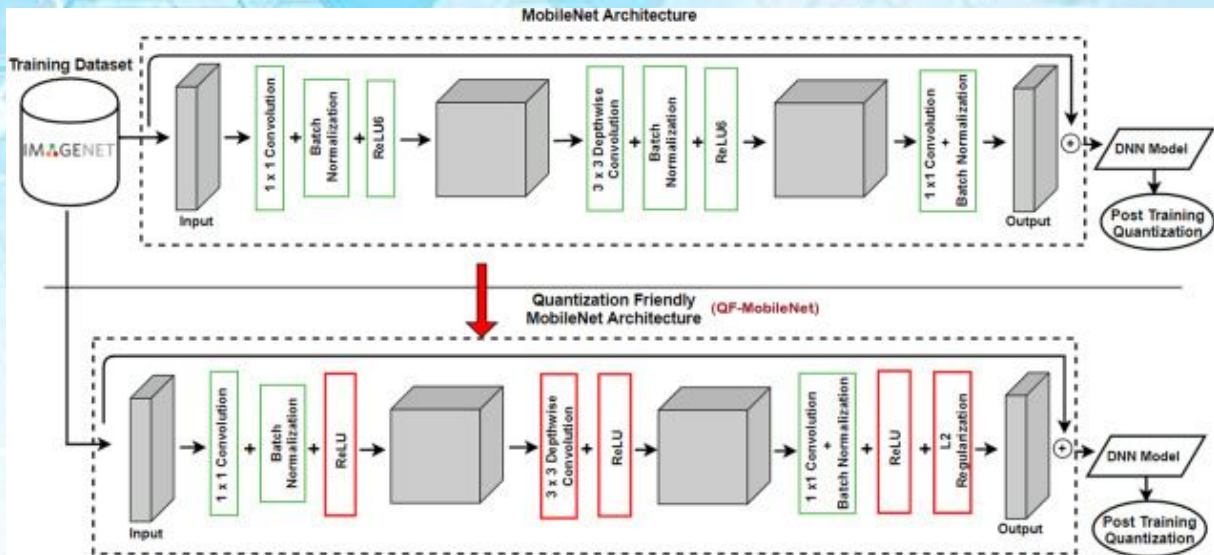
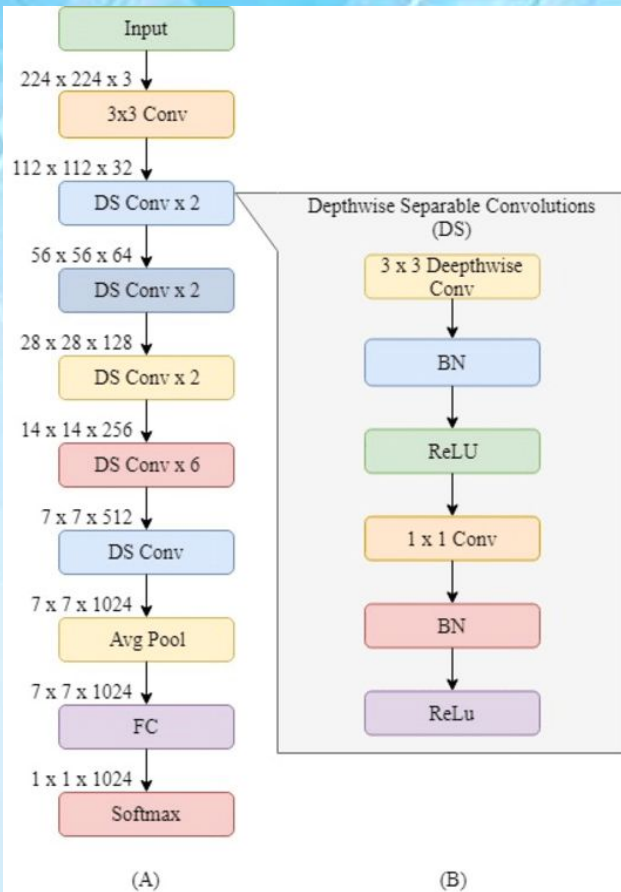
11. DenseNet



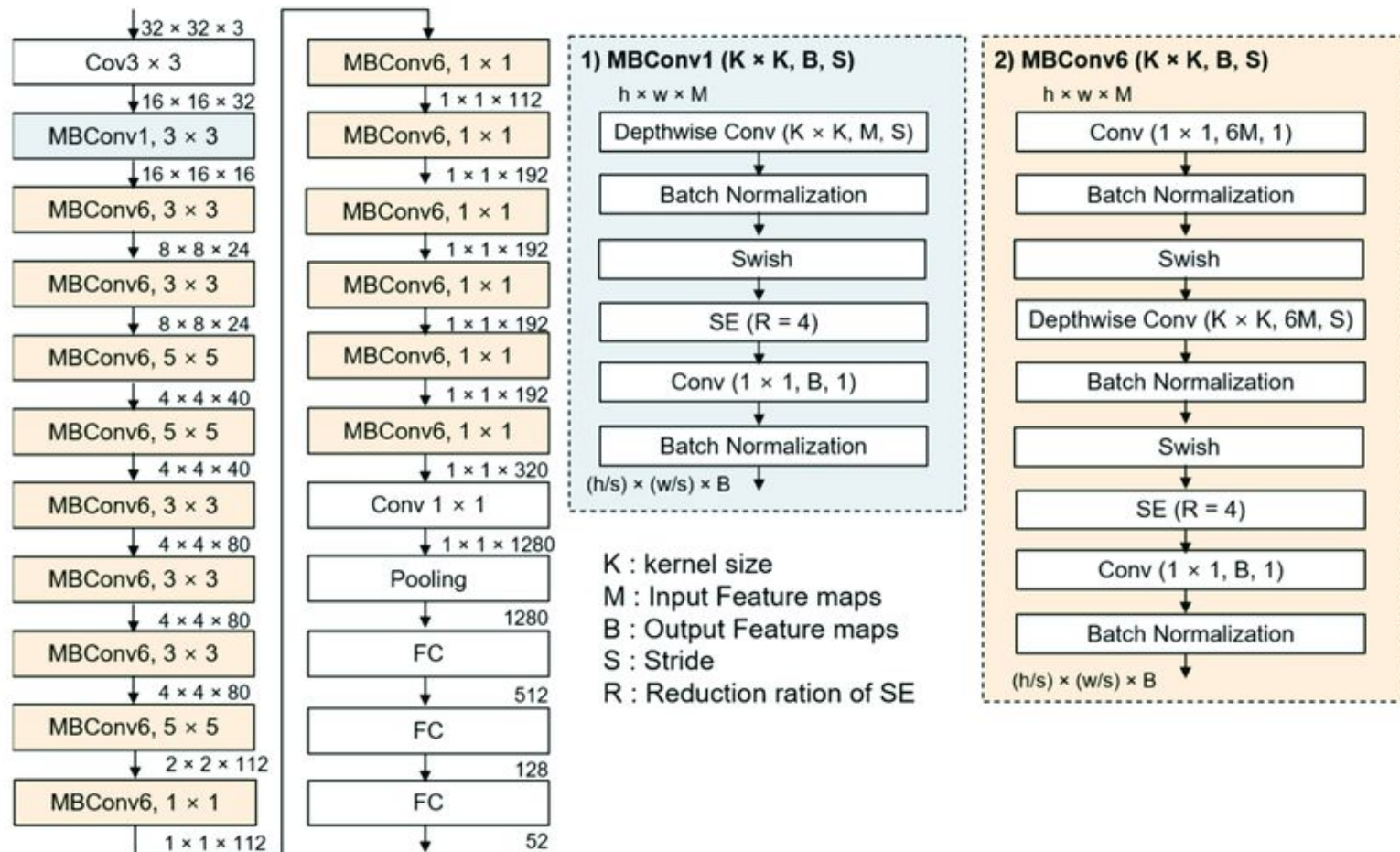
11. DenseNet



12. MobileNet v1



13. EfficientNet



CNNs for image classification with Pytorch

Object Detection From Scratch in PyTorch

<https://github.com/christopherbate/ObjectDetectionFromScratch/blob/master/README.md>

Object Detection YOLOs

Practice 5 -

Q & A